

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#04)

Use of Venus Simulator for Register Level Execution of RISC-V Arithmetic and Memory Instructions

Student Name: Muhammad Ashnab Safdar Roll No:24I-6500 Section: CE-A

Date performed: 8/9/2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

I used the Venus simulator to test basic RISC-V arithmetic instructions. I loaded two immediate values into registers with addi, then computed their sum, difference, product, and square using add, sub, and mul. I confirmed each destination register held the expected result after execution.

Code:

```
1 addi t0, x0, 5      # A = 5
2 addi t1, x0, 4      # B = 4
3 add  t2, t0, t1      # C = A + B
4 sub  t3, t0, t1      # D = A - B
5 mul  t4, t0, t1      # E = A * B
6 mul  t5, t0, t0      # F = A^2 (A*A)
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00500293	addi x5 x0 5	addi t0, x0, 5 # A = 5
0x4	0x00400313	addi x6 x0 4	addi t1, x0, 4 # B = 4
0x8	0x006283B3	add x7 x5 x6	add t2, t0, t1 # C = A + B
0xc	0x40628E33	sub x28 x5 x6	sub t3, t0, t1 # D = A - B
0x10	0x02628EB3	mul x29 x5 x6	mul t4, t0, t1 # E = A * B
0x14	0x02528F33	mul x30 x5 x5	mul t5, t0, t0 # F = A^2 (A*A)

t0 (x5)	0x00000005
t1 (x6)	0x00000004

t2 (x7)	0x00000009
t3 (x28)	0x00000001
t4 (x29)	0x00000014
t5 (x30)	0x00000019

Task 2:

Description:

I evaluated the compound expression $m = (a - c) + (d + e) \times f$ using RISC-V register instructions. I first computed the intermediate terms $(a - c)$ and $(d + e)$, then multiplied and added them together with sub, add, and mul. I checked the final value in t5 against the expected result to confirm correctness.

Code:

```
1 sub t6, t0, t1      # t6 = a - c
2 add t5, t2, t3      # t5 = d + e
3 mul t5, t5, t4      # t5 = (d + e) * f
4 add t5, t6, t5      # m = (a - c) + (d + e) * f
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x40628FB3	sub x31 x5 x6	sub t6, t0, t1 # t6 = a - c
0x4	0x01C38F33	add x30 x7 x28	add t5, t2, t3 # t5 = d + e
0x8	0x03DF0F33	mul x30 x30 x29	mul t5, t5, t4 # t5 = (d + e) * f
0xc	0x01EF8F33	add x30 x31 x30	add t5, t6, t5 # m = (a - c) + (d + e) * f

t6
(x31) 0x00000000

t5
(x30) 0x00000000

t5
(x30) 0x00000000

t5
(x30) 0x00000000

--

Task 3:

Description:
I implemented bitwise AND, OR, and XOR operations on two hexadecimal values, 0xFFF and 0xF0F, using RISC-V instructions. I stored each result in memory with sw and then printed all three values using the print-integer syscall (a7 = 1). I verified the outputs matched the expected bitwise results.
Code:

```
.data
result_and: .word 0
result_or:  .word 0
result_xor: .word 0

.text
.globl main

main:
    li    t0, 0xFFF          # t0 = 0xFFF
    li    t1, 0xF0F          # t1 = 0xF0F

    and   t2, t0, t1          # AND
    or    t3, t0, t1          # OR
    xor   t4, t0, t1          # XOR

    la    t5, result_and
    sw    t2, 0(t5)
    la    t5, result_or
    sw    t3, 0(t5)
    la    t5, result_xor
    sw    t4, 0(t5)

    li    a7, 1
    mv    a0, t2
    ecall
```

```
li    a7, 1
mv    a0, t3
ecall
```

```
li    a7, 1
mv    a0, t4
ecall
```

```
li    a0, 0
li    a7, 10
ecall
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x000012B7	lui x5 1	li t0, 0xFFF # t0 = 0xFFF
0x4	0xFFFF28293	addi x5 x5 -1	li t0, 0xFFF # t0 = 0xFFF
0x8	0x00001337	lui x6 1	li t1, 0xF0F # t1 = 0xF0F
0xc	0xF0F30313	addi x6 x6 -241	li t1, 0xF0F # t1 = 0xF0F
0x10	0x0062F3B3	and x7 x5 x6	and t2, t0, t1 # AND
0x14	0x0062EE33	or x28 x5 x6	or t3, t0, t1 # OR
0x18	0x0062CEB3	xor x29 x5 x6	xor t4, t0, t1 # XOR
0x1c	0x10000F17	auipc x30 65536	la t5, result_and
0x20	0xFE4F0F13	addi x30 x30 -28	la t5, result_and
0x24	0x007F2023	sw x7 0(x30)	sw t2, 0(t5)
0x28	0x10000F17	auipc x30 65536	la t5, result_or
0x2c	0xFD4F0F13	addi x30 x30 -36	la t5, result_or
0x30	0x01CF2023	sw x28 0(x30)	sw t3, 0(t5)
0x34	0x10000F17	auipc x30 65536	la t5, result_xor
0x38	0xFD4F0F13	addi x30 x30 -44	la t5, result_xor
0x3c	0x01DF2023	sw x29 0(x30)	sw t4, 0(t5)
0x40	0x00100893	addi x17 x0 1	li a7, 1
0x44	0x00038513	addi x10 x7 0	mv a0, t2
0x48	0x00000073	ecall	ecall
0x4c	0x00100893	addi x17 x0 1	li a7, 1
0x50	0x000E0513	addi x10 x28 0	mv a0, t3

0x48	0x00000073	ecall	ecall
0x4c	0x00100893	addi x17 x0 1	li a7, 1
0x50	0x000E0513	addi x10 x28 0	mv a0, t3
0x54	0x00000073	ecall	ecall
0x58	0x00100893	addi x17 x0 1	li a7, 1
0x5c	0x000E8513	addi x10 x29 0	mv a0, t4
0x60	0x00000073	ecall	ecall
0x64	0x00000513	addi x10 x0 0	li a0, 0
0x68	0x00A00893	addi x17 x0 10	li a7, 10
0x6c	0x00000073	ecall	ecall
t0 (x5)	0x00001000		
t0 (x5)	0x00000FFF		
t1 (x6)	0x00001000		
t1 (x6)	0x00000F0F		
t2 (x7)	0x00000F0F		
t3 (x28)	0x00000FFF		
t4 (x29)	0x000000F0		
t5 (x30)	0x1000001c		
t5 (x30)	0x10000000		

t5 (x30)	0x10000000
t5 (x30)	0x10000028
t5 (x30)	0x10000004
t5 (x30)	0x10000004
t5 (x30)	0x10000034
t5 (x30)	0x10000008
t5 (x30)	0x10000008
a7 (x17)	0x00000001
a0 (x10)	0x00000F0F
a0 (x10)	0x00000F0F
a7 (x17)	0x00000001
a7 (x17)	0x00000001
a0 (x10)	0x000000F0

a0 (x10)	0x000000F0
a0 (x10)	0x00000000
a7 (x17)	0x0000000A

Task 4:

Description:

I wrote a program that sums a 5-element integer array using a separate function, sumArray. I passed the array's base address and size through a0 and a1, called the function with jal, and used a loop with bge, slli, and lw to add each element to a running total in t3. I confirmed the returned sum in a0 matched the expected value of 15.

Code:

```
.data
numbers: .word 1, 2, 3, 4, 5
count: .word 5

.text

.globl main
main:

    la  a0, numbers    # array address

    lw  a1, count       # array size
```

```
jal ra, sumArray    # call sum function

li a0, 10           # exit program

ecall

sumArray:

li t3, 0            # start sum at 0

li t4, 0            # start index at 0

sum_loop:

bge t4, a1, sum_done # stop when index reaches size

slli t5, t4, 2       # calculate offset

add t5, a0, t5       # find element address

lw t5, 0(t5)         # load element

add t3, t3, t5        # add element to sum

addi t4, t4, 1        # move to next element

j sum_loop           # repeat loop

sum_done:

mv a0, t3            # put sum in a0

ret                  # return
```

Output:

Index (t4)	Address (t5)	Loaded	Sum (t3)
0	a0+0	1	1
1	a0+4	2	3
2	a0+8	3	6
3	a0+12	4	10
4	a0+16	5	15
5	exit	—	15

Task 5:

Description:

I loaded seven word values from a data table into the saved registers s1 through s7 using la and lw with fixed byte offsets. I used this task to practice base-plus-offset addressing for accessing array elements. I confirmed each register held the correct value by inspecting them in the simulator after execution.

Code:

```
.data

values: .word 0, 2, 4, 6, 8, 10, 12

.text

.globl main

main:

    la  s0, values    # table address

    lw  s1, 0(s0)     # load first value
```

```
lw s2, 4(s0)    # load second value

lw s3, 8(s0)    # load third value

lw s4, 12(s0)   # load fourth value

lw s5, 16(s0)   # load fifth value

lw s6, 20(s0)   # load sixth value

lw s7, 24(s0)   # load seventh value
```

```
li a0, 10      # exit program
```

```
ecall
```

Output:

```
s0  0x10000000
(x8)

(x8)
a0  0x10000000

s1  0x00000000
(x9)

(x18)
a5  0x00000005

s3  0x00000004
(x19)

s4  0x00000006
(x20)
```

s5 (x21)	0x00000008
s6 (x22)	0x0000000A
s7 (x23)	0x0000000C
a0 (x10)	0x0000000A